QuantiumTask1DataCleaning

February 27, 2026

1 Importing Files

[1]:
```
import pandas as pd;
import numpy as np;
import matplotlib.pyplot as plt;

customer_data = pd.read_csv("C:/Users/user/OneDrive - Hetauda School of␣
↪Management and Social Sciences/Desktop/Data Analysis/Projects/Project III/
↪Raw Data & Problems/QVI_purchase_behaviour.csv");
transaction_data = pd.read_excel("C:/Users/user/OneDrive - Hetauda School of␣
↪Management and Social Sciences/Desktop/Data Analysis/Projects/Project III/
↪Raw Data & Problems/QVI_transaction_data.xlsx");

print(transaction_data.head());
```

```
    DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0  43390          1            1000       1         5
1  43599          1            1307     348        66
2  43605          1            1343     383        61
3  43329          2            2373     974        69
4  43330          2            2426    1038       108

                                PROD_NAME  PROD_QTY  TOT_SALES
0    Natural Chip        Compny SeaSalt175g         2        6.0
1                  CCs Nacho Cheese    175g         3        6.3
2    Smiths Crinkle Cut  Chips Chicken 170g         2        2.9
3    Smiths Chip Thinly  S/Cream&Onion 175g         5       15.0
4  Kettle Tortilla ChpsHny&Jlpno Chili 150g         3       13.8
```

[2]:
```
print(customer_data.head());
```

```
   LYLTY_CARD_NBR               LIFESTAGE PREMIUM_CUSTOMER
0            1000   YOUNG SINGLES/COUPLES          Premium
1            1002   YOUNG SINGLES/COUPLES       Mainstream
2            1003          YOUNG FAMILIES           Budget
3            1004   OLDER SINGLES/COUPLES       Mainstream
4            1005  MIDAGE SINGLES/COUPLES       Mainstream
```

[3]:
```
print(transaction_data.describe());
```

```
                DATE     STORE_NBR  LYLTY_CARD_NBR        TXN_ID  \
count  264836.000000  264836.00000    2.648360e+05  2.648360e+05
mean    43464.036260     135.08011    1.355495e+05  1.351583e+05
std       105.389282      76.78418    8.057998e+04  7.813303e+04
min     43282.000000       1.00000    1.000000e+03  1.000000e+00
25%     43373.000000      70.00000    7.002100e+04  6.760150e+04
50%     43464.000000     130.00000    1.303575e+05  1.351375e+05
75%     43555.000000     203.00000    2.030942e+05  2.027012e+05
max     43646.000000     272.00000    2.373711e+06  2.415841e+06

            PROD_NBR       PROD_QTY      TOT_SALES
count  264836.000000  264836.000000  264836.000000
mean       56.583157       1.907309       7.304200
std        32.826638       0.643654       3.083226
min         1.000000       1.000000       1.500000
25%        28.000000       2.000000       5.400000
50%        56.000000       2.000000       7.400000
75%        85.000000       2.000000       9.200000
max       114.000000     200.000000     650.000000
```

[4]:
```
print(transaction_data.dtypes);
```

```
DATE                int64
STORE_NBR           int64
LYLTY_CARD_NBR      int64
TXN_ID              int64
PROD_NBR            int64
PROD_NAME          object
PROD_QTY            int64
TOT_SALES         float64
dtype: object
```

2 Changing Date Format

[5]:
```
if(transaction_data["DATE"].dtype != "datetime64[ns]"):
        transaction_data["DATE"] = pd.to_datetime(transaction_data["DATE"] ,␣
 ↪origin="1899-12-30" , unit="D");
print(transaction_data.dtypes);
```

```
DATE              datetime64[ns]
STORE_NBR                  int64
LYLTY_CARD_NBR             int64
TXN_ID                     int64
PROD_NBR                   int64
PROD_NAME                 object
PROD_QTY                   int64
```

```
TOT_SALES                float64
dtype: object
```

[6]:
```python
print(transaction_data.head());
```

```
        DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0 2018-10-17          1            1000       1         5
1 2019-05-14          1            1307     348        66
2 2019-05-20          1            1343     383        61
3 2018-08-17          2            2373     974        69
4 2018-08-18          2            2426    1038       108

                                  PROD_NAME  PROD_QTY  TOT_SALES
0    Natural Chip        Compny SeaSalt175g         2        6.0
1                  CCs Nacho Cheese    175g         3        6.3
2    Smiths Crinkle Cut  Chips Chicken 170g         2        2.9
3    Smiths Chip Thinly  S/Cream&Onion 175g         5       15.0
4  Kettle Tortilla ChpsHny&Jlpno Chili 150g         3       13.8
```

3 Checking Nulls

[7]:
```python
print(transaction_data.isnull().sum());
```

```
DATE              0
STORE_NBR         0
LYLTY_CARD_NBR    0
TXN_ID            0
PROD_NBR          0
PROD_NAME         0
PROD_QTY          0
TOT_SALES         0
dtype: int64
```

[8]:
```python
print(transaction_data["PROD_NAME"].describe());
```

```
count                                   264836
unique                                     114
top       Kettle Mozzarella   Basil & Pesto 175g
freq                                      3304
Name: PROD_NAME, dtype: object
```

[9]:
```python
print(transaction_data["PROD_NAME"].unique());
```

```
['Natural Chip        Compny SeaSalt175g' 'CCs Nacho Cheese    175g'
 'Smiths Crinkle Cut  Chips Chicken 170g'
 'Smiths Chip Thinly  S/Cream&Onion 175g'
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g'
 'Old El Paso Salsa   Dip Tomato Mild 300g'
 'Smiths Crinkle Chips Salt & Vinegar 330g'
```

```
'Grain Waves         Sweet Chilli 210g'
'Doritos Corn Chip Mexican Jalapeno 150g'
'Grain Waves Sour    Cream&Chives 210G'
'Kettle Sensations   Siracha Lime 150g' 'Twisties Cheese     270g'
'WW Crinkle Cut      Chicken 175g' 'Thins Chips Light&  Tangy 175g'
'CCs Original 175g' 'Burger Rings 220g'
'NCC Sour Cream &    Garden Chives 175g'
'Doritos Corn Chip Southern Chicken 150g' 'Cheezels Cheese Box 125g'
'Smiths Crinkle      Original 330g'
'Infzns Crn Crnchers Tangy Gcamole 110g'
'Kettle Sea Salt     And Vinegar 175g'
'Smiths Chip Thinly  Cut Original 175g' 'Kettle Original 175g'
'Red Rock Deli Thai  Chilli&Lime 150g' 'Pringles Sthrn FriedChicken 134g'
'Pringles Sweet&Spcy BBQ 134g' 'Red Rock Deli SR    Salsa & Mzzrlla 150g'
'Thins Chips         Originl saltd 175g'
'Red Rock Deli Sp    Salt & Truffle 150G'
'Smiths Thinly       Swt Chli&S/Cream175G' 'Kettle Chilli 175g'
'Doritos Mexicana    170g' 'Smiths Crinkle Cut  French OnionDip 150g'
'Natural ChipCo      Hony Soy Chckn175g'
'Dorito Corn Chp     Supreme 380g' 'Twisties Chicken270g'
'Smiths Thinly Cut   Roast Chicken 175g'
'Smiths Crinkle Cut  Tomato Salsa 150g'
'Kettle Mozzarella   Basil & Pesto 175g'
'Infuzions Thai SweetChili PotatoMix 110g'
'Kettle Sensations   Camembert & Fig 150g'
'Smith Crinkle Cut   Mac N Cheese 150g'
'Kettle Honey Soy    Chicken 175g' 'Thins Chips Seasonedchicken 175g'
'Smiths Crinkle Cut  Salt & Vinegar 170g'
'Infuzions BBQ Rib   Prawn Crackers 110g'
'GrnWves Plus Btroot & Chilli Jam 180g'
'Tyrrells Crisps     Lightly Salted 165g'
'Kettle Sweet Chilli And Sour Cream 175g'
'Doritos Salsa       Medium 300g' 'Kettle 135g Swt Pot Sea Salt'
'Pringles SourCream  Onion 134g' 'Doritos Corn Chips  Original 170g'
'Twisties Cheese     Burger 250g'
'Old El Paso Salsa   Dip Chnky Tom Ht300g'
'Cobs Popd Swt/Chlli &Sr/Cream Chips 110g'
'Woolworths Mild     Salsa 300g'
'Natural Chip Co     Tmato Hrb&Spce 175g'
'Smiths Crinkle Cut  Chips Original 170g'
'Cobs Popd Sea Salt  Chips 110g'
'Smiths Crinkle Cut  Chips Chs&Onion170g'
'French Fries Potato Chips 175g'
'Old El Paso Salsa   Dip Tomato Med 300g'
'Doritos Corn Chips  Cheese Supreme 170g'
'Pringles Original   Crisps 134g' 'RRD Chilli&         Coconut 150g'
'WW Original Corn    Chips 200g' 'Thins Potato Chips  Hot & Spicy 175g'
'Cobs Popd Sour Crm  &Chives Chips 110g'
```

```
 'Smiths Crnkle Chip  Orgnl Big Bag 380g'
 'Doritos Corn Chips  Nacho Cheese 170g'
 'Kettle Sensations   BBQ&Maple 150g' 'WW D/Style Chip     Sea Salt 200g'
 'Pringles Chicken    Salt Crips 134g' 'WW Original Stacked Chips 160g'
 'Smiths Chip Thinly  CutSalt/Vinegr175g' 'Cheezels Cheese 330g'
 'Tostitos Lightly    Salted 175g' 'Thins Chips Salt &  Vinegar 175g'
 'Smiths Crinkle Cut  Chips Barbecue 170g' 'Cheetos Puffs 165g'
 'RRD Sweet Chilli &  Sour Cream 165g' 'WW Crinkle Cut      Original 175g'
 'Tostitos Splash Of  Lime 175g' 'Woolworths Medium   Salsa 300g'
 'Kettle Tortilla ChpsBtroot&Ricotta 150g' 'CCs Tasty Cheese    175g'
 'Woolworths Cheese   Rings 190g' 'Tostitos Smoked     Chipotle 175g'
 'Pringles Barbeque   134g' 'WW Supreme Cheese   Corn Chips 200g'
 'Pringles Mystery    Flavour 134g'
 'Tyrrells Crisps     Ched & Chives 165g'
 'Snbts Whlgrn Crisps Cheddr&Mstrd 90g' 'Cheetos Chs & Bacon Balls 190g'
 'Pringles Slt Vingar 134g' 'Infuzions SourCream&Herbs Veg Strws 110g'
 'Kettle Tortilla ChpsFeta&Garlic 150g'
 'Infuzions Mango     Chutny Papadums 70g'
 'RRD Steak &         Chimuchurri 150g' 'RRD Honey Soy       Chicken 165g'
 'Sunbites Whlegrn    Crisps Frch/Onin 90g' 'RRD Salt & Vinegar  165g'
 'Doritos Cheese      Supreme 330g' 'Smiths Crinkle Cut  Snag&Sauce 150g'
 'WW Sour Cream &OnionStacked Chips 160g' 'RRD Lime & Pepper   165g'
 'Natural ChipCo Sea  Salt & Vinegr 175g'
 'Red Rock Deli Chikn&Garlic Aioli 150g'
 'RRD SR Slow Rst     Pork Belly 150g' 'RRD Pc Sea Salt     165g'
 'Smith Crinkle Cut   Bolognese 150g' 'Doritos Salsa Mild  300g']
```

4 Splitting every words of products.

[10]:
```
unique_products = transaction_data["PROD_NAME"].unique();

allwords = [];

for product in unique_products:
    allwords.extend(product.split());

words_df = pd.DataFrame(allwords , columns=["Words"]);
print(words_df.tail(6));
```

```
             Words
529            90g
530            RRD
531           Salt
532              &
533        Vinegar
534           165g
535        Doritos
```

```
536         Cheese
537        Supreme
538           330g
539         Smiths
540        Crinkle
541            Cut
542     Snag&Sauce
543           150g
544             WW
545           Sour
546          Cream
547  &OnionStacked
548          Chips
549           160g
550            RRD
551           Lime
552              &
553         Pepper
554           165g
555        Natural
556         ChipCo
557            Sea
558           Salt
559              &
560         Vinegr
561           175g
562            Red
563           Rock
564           Deli
565   Chikn&Garlic
566          Aioli
567           150g
568            RRD
569             SR
570           Slow
571            Rst
572           Pork
573          Belly
574           150g
575            RRD
576             Pc
577            Sea
578           Salt
579           165g
580          Smith
581        Crinkle
582            Cut
583      Bolognese
```

```
584          150g
585       Doritos
586         Salsa
587          Mild
588          300g
```

5 Remove Numeric Characters

[11]:
```python
words_df = words_df[~words_df["Words"].str.contains(r"\d")];
print(words_df.tail(5));
```

```
            Words
527         Crisps
528      Frch/Onin
530            RRD
531           Salt
532              &
533        Vinegar
535        Doritos
536         Cheese
537        Supreme
539         Smiths
540        Crinkle
541            Cut
542     Snag&Sauce
544             WW
545           Sour
546          Cream
547  &OnionStacked
548          Chips
550            RRD
551           Lime
552              &
553         Pepper
555        Natural
556         ChipCo
557            Sea
558           Salt
559              &
560         Vinegr
562            Red
563           Rock
564           Deli
565   Chikn&Garlic
566          Aioli
568            RRD
569             SR
```

```
570           Slow
571            Rst
572           Pork
573          Belly
575            RRD
576             Pc
577            Sea
578           Salt
580          Smith
581        Crinkle
582            Cut
583      Bolognese
585        Doritos
586          Salsa
587           Mild
```

6 Remove Special Characters like &.

[12]:
```python
words_df = words_df[words_df["Words"].str.isalpha()];
print(words_df.tail(6));
```

```
           Words
503          Veg
504        Strws
506       Kettle
507     Tortilla
510    Infuzions
511        Mango
512       Chutny
513     Papadums
515          RRD
516        Steak
518  Chimuchurri
520          RRD
521        Honey
522          Soy
523      Chicken
525     Sunbites
526      Whlegrn
527       Crisps
530          RRD
531         Salt
533      Vinegar
535      Doritos
536       Cheese
537      Supreme
539       Smiths
```

```
540      Crinkle
541          Cut
544           WW
545         Sour
546        Cream
548        Chips
550          RRD
551         Lime
553       Pepper
555      Natural
556       ChipCo
557          Sea
558         Salt
560       Vinegr
562          Red
563         Rock
564         Deli
566        Aioli
568          RRD
569           SR
570         Slow
571          Rst
572         Pork
573        Belly
575          RRD
576           Pc
577          Sea
578         Salt
580        Smith
581      Crinkle
582          Cut
583    Bolognese
585      Doritos
586        Salsa
587         Mild
```

[13]:
```python
#lowercasing every words
words_df  = words_df["Words"].str.lower();
print(words_df.tail());
```

```
582          cut
583    bolognese
585      doritos
586        salsa
587         mild
Name: Words, dtype: object
```

[14]:
```
count_salsa = (words_df.values == "salsa").sum();
count_dip = (words_df.values == "dip").sum();
count_dips = (words_df.values == "dips").sum();
count_sauce = (words_df.values == "sauce").sum();
print(count_salsa);
print(count_dip);
print(count_dips);
print(count_sauce);
```

```
9
3
0
0
```

7 Removing Salsa

[15]:
```
transaction_data['SALSA'] = transaction_data['PROD_NAME'].str.lower().str.
 ↪contains(r'\bsalsa\b');

before_count = transaction_data['PROD_NAME'].str.lower().str.
 ↪contains(r'\bsalsa\b').sum();
print("Rows containing salsa before removal:", before_count);

transaction_data = transaction_data[transaction_data['SALSA'] == False];

#Removing unnecessary column named salsa after operation.
transaction_data = transaction_data.drop(columns=['SALSA']);

after_count = transaction_data['PROD_NAME'].str.lower().str.
 ↪contains(r'\bsalsa\b').sum();
print("Rows containing salsa after removal:", after_count);
```

```
Rows containing salsa before removal: 18094
Rows containing salsa after removal: 0
```

8 Removing Dip

[16]:
```
transaction_data["DipCheck"] = transaction_data["PROD_NAME"].str.lower().str.
 ↪contains(r"\bdip\b") ;
count_before = transaction_data["PROD_NAME"].str.lower().str.contains(r"\bdips?
 ↪\b").sum() ;
transaction_data = transaction_data.drop(columns=['DipCheck']);
print("Before dip count:" , count_before);
```

```
Before dip count: 0
```

9 Removing Numeric Characters and special keywords.

[17]:
```
print('Before' , transaction_data["PROD_NAME"].tail(20));
transaction_data["Packet_Wt"] = transaction_data["PROD_NAME"].str.split().
 ↪str[-1].str.replace(r"[^0-9]" , "" , regex=True);
transaction_data["PROD_NAME"] = (
    transaction_data["PROD_NAME"]
    .str.lower()
    .str.replace(r"&", "and", regex=False) #replace & with and
    .str.replace(r"[^a-z\s]", "", regex=True) #remove numbers & special chars
    .str.replace(r"\bg\b", "", regex=True) #remove 'g'
    .str.replace(r"\s+", " ", regex=True) #remove extra spaces
    .str.strip() #remove leading/trailing spaces
)
print('After',transaction_data["PROD_NAME"].tail());
```

```
Before 264816              Cobs Popd Sea Salt  Chips 110g
264817    Sunbites Whlegrn    Crisps Frch/Onin 90g
264818    Infuzions SourCream&Herbs Veg Strws 110g
264819                       Kettle Original 175g
264820            Pringles Mystery    Flavour 134g
264821        Kettle Sea Salt     And Vinegar 175g
264822                    Pringles Slt Vingar 134g
264823                Kettle 135g Swt Pot Sea Salt
264824     Kettle Tortilla ChpsBtroot&Ricotta 150g
264825     Infuzions BBQ Rib   Prawn Crackers 110g
264826              Cobs Popd Sea Salt  Chips 110g
264827    Infuzions Thai SweetChili PotatoMix 110g
264828         Pringles Chicken    Salt Crips 134g
264829    Infuzions SourCream&Herbs Veg Strws 110g
264830              Thins Chips Light&  Tangy 175g
264831     Kettle Sweet Chilli And Sour Cream 175g
264832               Tostitos Splash Of  Lime 175g
264833                    Doritos Mexicana    170g
264834     Doritos Corn Chip Mexican Jalapeno 150g
264835               Tostitos Splash Of  Lime 175g
Name: PROD_NAME, dtype: object
After 264816                 cobs popd sea salt chips
264817         sunbites whlegrn crisps frchonin
264818    infuzions sourcreamandherbs veg strws
264819                          kettle original
264820                 pringles mystery flavour
264821              kettle sea salt and vinegar
264822                      pringles slt vingar
264823                 kettle swt pot sea salt
264824     kettle tortilla chpsbtrootandricotta
264825         infuzions bbq rib prawn crackers
264826                 cobs popd sea salt chips
```

```
264827      infuzions thai sweetchili potatomix
264828              pringles chicken salt crips
264829    infuzions sourcreamandherbs veg strws
264830               thins chips lightand tangy
264831       kettle sweet chilli and sour cream
264832                  tostitos splash of lime
264833                         doritos mexicana
264834       doritos corn chip mexican jalapeno
264835                  tostitos splash of lime
Name: PROD_NAME, dtype: object
```

[18]:
```
print(transaction_data.head());
```

```
        DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0 2018-10-17          1            1000       1         5
1 2019-05-14          1            1307     348        66
2 2019-05-20          1            1343     383        61
3 2018-08-17          2            2373     974        69
4 2018-08-18          2            2426    1038       108

                              PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt
0           natural chip compny seasaltg         2        6.0       175
1                       ccs nacho cheese         3        6.3       175
2       smiths crinkle cut chips chicken         2        2.9       170
3      smiths chip thinly screamandonion         5       15.0       175
4  kettle tortilla chpshnyandjlpno chili         3       13.8       150
```

10 Checking the outliers.

[19]:
```
print("Store Number -:\n", transaction_data["STORE_NBR"].describe()) ;
print("Product Number -:\n", transaction_data["PROD_NBR"].describe()) ;
print("Product quantity -:\n", transaction_data["PROD_QTY"].describe()) ;
print("Sales -:\n", transaction_data["TOT_SALES"].describe()) ;
```

```
Store Number -:
 count    246742.000000
mean        135.051098
std          76.787096
min           1.000000
25%          70.000000
50%         130.000000
75%         203.000000
max         272.000000
Name: STORE_NBR, dtype: float64
Product Number -:
 count    246742.000000
mean         56.351789
std          33.695428
```

```
min           1.000000
25%          26.000000
50%          53.000000
75%          87.000000
max         114.000000
Name: PROD_NBR, dtype: float64
Product quantity -:
 count    246742.000000
mean          1.908062
std           0.659831
min           1.000000
25%           2.000000
50%           2.000000
75%           2.000000
max         200.000000
Name: PROD_QTY, dtype: float64
Sales -:
 count    246742.000000
mean          7.321322
std           3.077828
min           1.700000
25%           5.800000
50%           7.400000
75%           8.800000
max         650.000000
Name: TOT_SALES, dtype: float64
```

11 checking outliers count

[20]:
```
print(transaction_data['PROD_QTY'].value_counts().sort_index());
print(transaction_data['TOT_SALES'].value_counts().sort_index());
```

```
PROD_QTY
1       25476
2      220070
3         408
4         371
5         415
200         2
Name: count, dtype: int64
TOT_SALES
1.7      708
1.8      175
1.9      930
2.1      732
2.3      186
        …
```

```
23.6       4
27.0      35
28.5      12
29.5       7
650.0      2
Name: count, Length: 105, dtype: int64
```

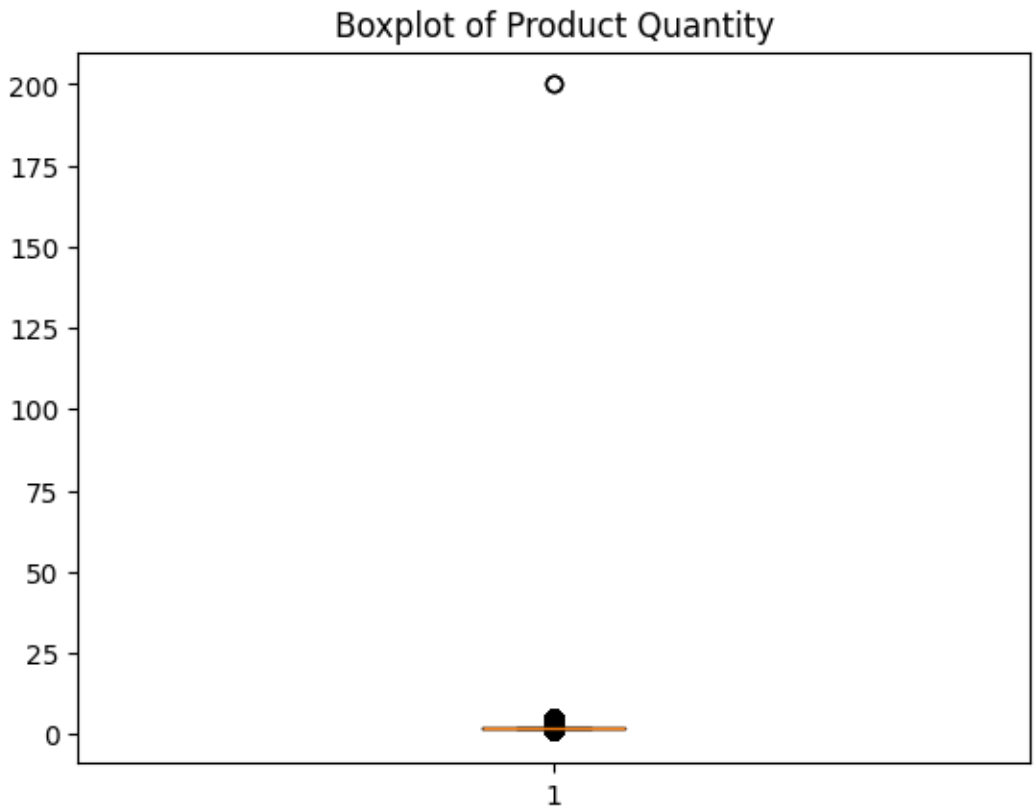
[21]:
```
outlier_transaction = transaction_data[transaction_data["PROD_QTY"]==200] ;
outlier_transaction
```

[21]:
```
            DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
69762 2018-08-19        226          226000  226201         4
69763 2019-05-20        226          226000  226210         4

                     PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt
69762  dorito corn chp supreme       200      650.0       380
69763  dorito corn chp supreme       200      650.0       380
```

[22]:
```
customer_id = outlier_transaction['LYLTY_CARD_NBR'].iloc[0]

customer_transactions = transaction_data[
    transaction_data['LYLTY_CARD_NBR'] == customer_id
]

customer_transactions
```

[22]:
```
            DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
69762 2018-08-19        226          226000  226201         4
69763 2019-05-20        226          226000  226210         4

                     PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt
69762  dorito corn chp supreme       200      650.0       380
69763  dorito corn chp supreme       200      650.0       380
```

[23]:
```
plt.boxplot(transaction_data['PROD_QTY'])
plt.title("Boxplot of Product Quantity")
plt.show()
```



12 Removing the outlier(Bulk Purchase) column.

[24]:
```
transaction_data = transaction_data[transaction_data["PROD_QTY"] != 200];
transaction_data["PROD_QTY"].value_counts().sort_index()
```

[24]:
```
PROD_QTY
1     25476
2    220070
3       408
4       371
5       415
Name: count, dtype: int64
```

[25]:
```
transactions_by_day = (
    transaction_data.groupby("DATE")
    .size() #How many rows are there in each DATE Group
    .reset_index(name="Number of transaction")
)

transactions_by_day.head()
```

[25]:
```
         DATE  Number of transaction
0 2018-07-01                    663
1 2018-07-02                    650
2 2018-07-03                    674
3 2018-07-04                    669
4 2018-07-05                    660
```

[26]:
```python
transactions_by_day.tail()
```

[26]:
```
           DATE  Number of transaction
359 2019-06-26                    657
360 2019-06-27                    669
361 2019-06-28                    673
362 2019-06-29                    703
363 2019-06-30                    704
```

13 Ensuring there is no date left even if there is 0 transactions.

[27]:
```python
full_dates = pd.date_range(start="2018-07-01", end="2019-06-30")
full_dates_df = pd.DataFrame({"DATE": full_dates})

transactions_by_day_full = full_dates_df.merge(transactions_by_day, on="DATE",␣
↪how="left")

transactions_by_day_full
```

[27]:
```
           DATE  Number of transaction
0   2018-07-01                  663.0
1   2018-07-02                  650.0
2   2018-07-03                  674.0
3   2018-07-04                  669.0
4   2018-07-05                  660.0
..         …                    …
360 2019-06-26                  657.0
361 2019-06-27                  669.0
362 2019-06-28                  673.0
363 2019-06-29                  703.0
364 2019-06-30                  704.0

[365 rows x 2 columns]
```

14 Filling null as 0

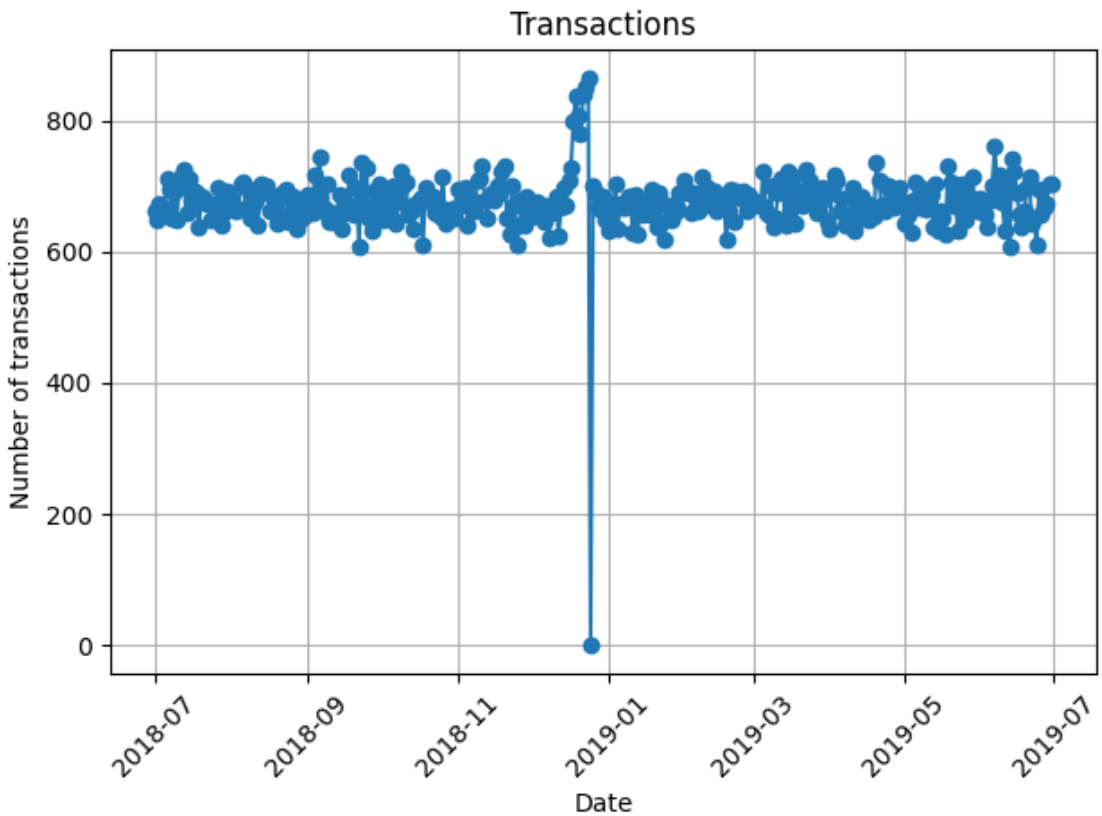
[28]:
```
transactions_by_day_full["Number of transaction"] =␣
↪transactions_by_day_full["Number of transaction"].fillna(0)

transactions_by_day_full.head()
```

[28]:

	DATE	Number of transaction
0	2018-07-01	663.0
1	2018-07-02	650.0
2	2018-07-03	674.0
3	2018-07-04	669.0
4	2018-07-05	660.0
5	2018-07-06	711.0
6	2018-07-07	695.0
7	2018-07-08	653.0
8	2018-07-09	692.0
9	2018-07-10	650.0
10	2018-07-11	701.0
11	2018-07-12	717.0
12	2018-07-13	727.0
13	2018-07-14	661.0
14	2018-07-15	712.0
15	2018-07-16	678.0
16	2018-07-17	694.0
17	2018-07-18	689.0
18	2018-07-19	637.0
19	2018-07-20	684.0
20	2018-07-21	683.0
21	2018-07-22	673.0
22	2018-07-23	673.0
23	2018-07-24	648.0
24	2018-07-25	674.0
25	2018-07-26	672.0
26	2018-07-27	697.0
27	2018-07-28	640.0
28	2018-07-29	659.0
29	2018-07-30	692.0
30	2018-07-31	688.0
31	2018-08-01	680.0
32	2018-08-02	669.0
33	2018-08-03	662.0
34	2018-08-04	665.0
35	2018-08-05	705.0
36	2018-08-06	706.0
37	2018-08-07	668.0
38	2018-08-08	695.0

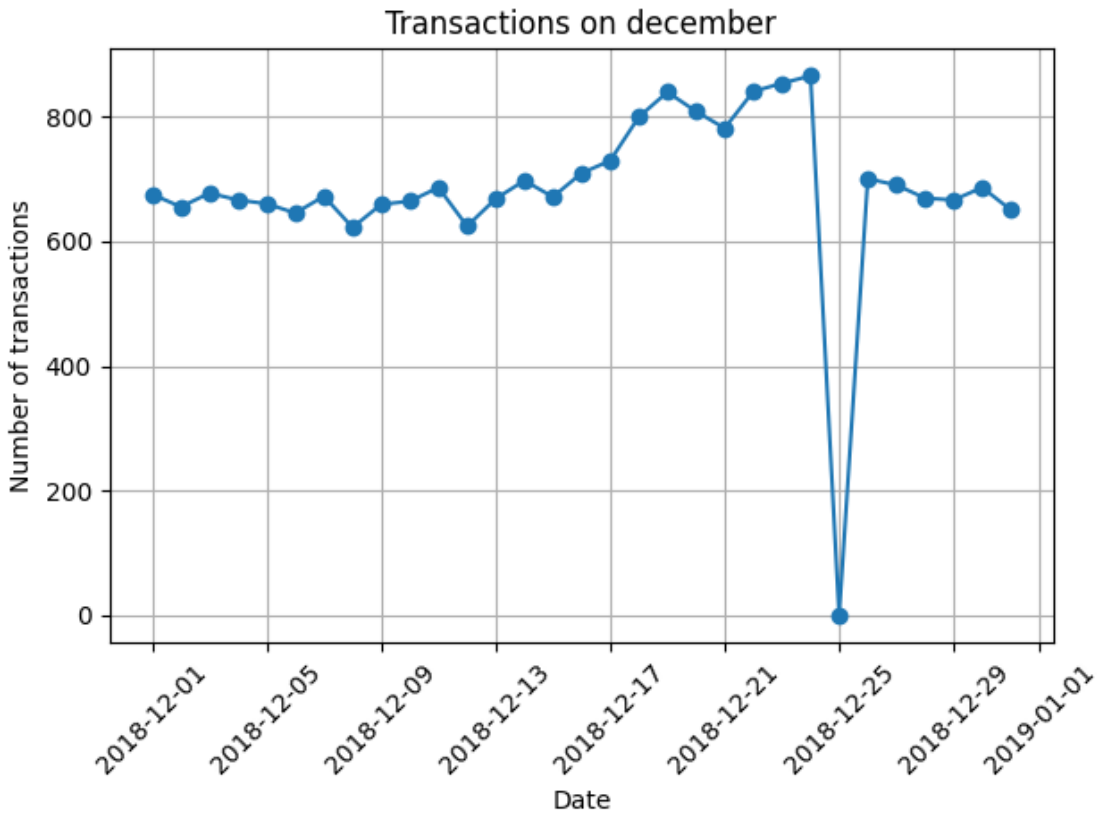
```
39 2018-08-09                  652.0
40 2018-08-10                  675.0
41 2018-08-11                  678.0
42 2018-08-12                  642.0
43 2018-08-13                  703.0
44 2018-08-14                  702.0
45 2018-08-15                  702.0
46 2018-08-16                  690.0
47 2018-08-17                  663.0
48 2018-08-18                  683.0
49 2018-08-19                  670.0
50 2018-08-20                  644.0
51 2018-08-21                  653.0
52 2018-08-22                  689.0
53 2018-08-23                  696.0
54 2018-08-24                  647.0
```

[29]:
```
plt.plot(transactions_by_day_full["DATE"], transactions_by_day_full["Number of␣
↪transaction"], marker='o', linestyle='-')
plt.title("Transactions")
plt.xlabel("Date")
plt.ylabel("Number of transactions")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout() #To avoid overlap
plt.show()
```



15 Plotting the dim in christmas

[30]:
```
dec_data = transactions_by_day_full[
    (transactions_by_day_full["DATE"] >= "2018-12-01") &
    (transactions_by_day_full["DATE"] <= "2018-12-31")
]

plt.plot(dec_data["DATE"], dec_data["Number of transaction"], marker='o')
plt.title("Transactions on december")
plt.xlabel("Date")
plt.ylabel("Number of transactions")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout() #To avoid overlap
plt.show()
```



[31]:
```
transaction_data.head()
```

[31]:
```
        DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0 2018-10-17          1            1000       1         5
1 2019-05-14          1            1307     348        66
2 2019-05-20          1            1343     383        61
3 2018-08-17          2            2373     974        69
4 2018-08-18          2            2426    1038       108

                              PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt
0           natural chip compny seasaltg         2        6.0       175
1                       ccs nacho cheese         3        6.3       175
2       smiths crinkle cut chips chicken         2        2.9       170
3      smiths chip thinly screamandonion         5       15.0       175
4  kettle tortilla chpshnyandjlpno chili         3       13.8       150
```

16 Splitting the brand name

[32]:
```
transaction_data["BRANDS"] = transaction_data["PROD_NAME"].str.split().str[0] ;
transaction_data[["PROD_NAME","BRANDS"]].head()
```

[32]:

	PROD_NAME	BRANDS
0	natural chip compny seasaltg	natural
1	ccs nacho cheese	ccs
2	smiths crinkle cut chips chicken	smiths
3	smiths chip thinly screamandonion	smiths
4	kettle tortilla chpshnyandjlpno chili	kettle
6	smiths crinkle chips salt and vinegar	smiths
7	grain waves sweet chilli	grain
8	doritos corn chip mexican jalapeno	doritos
9	grain waves sour creamandchives	grain
10	smiths crinkle chips salt and vinegar	smiths
11	kettle sensations siracha lime	kettle
12	twisties cheese	twisties
13	ww crinkle cut chicken	ww
14	thins chips lightand tangy	thins
15	ccs original	ccs
16	burger rings	burger
17	ncc sour cream and garden chives	ncc
18	doritos corn chip southern chicken	doritos
19	cheezels cheese box	cheezels
20	smiths crinkle original	smiths

[33]:
```
transaction_data['BRANDS'].unique()
```

[33]:
```
array(['natural', 'ccs', 'smiths', 'kettle', 'grain', 'doritos',
       'twisties', 'ww', 'thins', 'burger', 'ncc', 'cheezels', 'infzns',
       'red', 'pringles', 'dorito', 'infuzions', 'smith', 'grnwves',
       'tyrrells', 'cobs', 'french', 'rrd', 'tostitos', 'cheetos',
       'woolworths', 'snbts', 'sunbites'], dtype=object)
```

17 Solving typo in Brands

[34]:
```
transaction_data["BRANDS"] = transaction_data["BRANDS"].replace({"smith":
 ↪"smiths"});
transaction_data["BRANDS"] = transaction_data["BRANDS"].replace({"red":"rrd"});
transaction_data['BRANDS'].unique()
```

[34]:
```
array(['natural', 'ccs', 'smiths', 'kettle', 'grain', 'doritos',
       'twisties', 'ww', 'thins', 'burger', 'ncc', 'cheezels', 'infzns',
       'rrd', 'pringles', 'dorito', 'infuzions', 'grnwves', 'tyrrells',
       'cobs', 'french', 'tostitos', 'cheetos', 'woolworths', 'snbts',
       'sunbites'], dtype=object)
```

[35]:
```
transaction_data.head()
```

[35]:
```
        DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0 2018-10-17          1            1000       1         5
1 2019-05-14          1            1307     348        66
2 2019-05-20          1            1343     383        61
3 2018-08-17          2            2373     974        69
4 2018-08-18          2            2426    1038       108

                              PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt  \
0           natural chip compny seasaltg         2        6.0       175
1                       ccs nacho cheese         3        6.3       175
2       smiths crinkle cut chips chicken         2        2.9       170
3      smiths chip thinly screamandonion         5       15.0       175
4  kettle tortilla chpshnyandjlpno chili         3       13.8       150

    BRANDS
0  natural
1      ccs
2   smiths
3   smiths
4   kettle
```

[36]:
```
customer_data.head()
```

[36]:
```
   LYLTY_CARD_NBR               LIFESTAGE PREMIUM_CUSTOMER
0            1000   YOUNG SINGLES/COUPLES          Premium
1            1002   YOUNG SINGLES/COUPLES       Mainstream
2            1003          YOUNG FAMILIES           Budget
3            1004   OLDER SINGLES/COUPLES       Mainstream
4            1005  MIDAGE SINGLES/COUPLES       Mainstream
```

18 merging two datas.

[39]:
```
data = pd.merge(transaction_data, customer_data, on="LYLTY_CARD_NBR",␣
 ↪how="left");
data.head()
```

[39]:
```
        DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0 2018-10-17          1            1000       1         5
1 2019-05-14          1            1307     348        66
2 2019-05-20          1            1343     383        61
3 2018-08-17          2            2373     974        69
4 2018-08-18          2            2426    1038       108

                              PROD_NAME  PROD_QTY  TOT_SALES Packet_Wt  \
0           natural chip compny seasaltg         2        6.0       175
```

```
1                       ccs nacho cheese         3        6.3       175
2       smiths crinkle cut chips chicken         2        2.9       170
3      smiths chip thinly screamandonion         5       15.0       175
4  kettle tortilla chpshnyandjlpno chili         3       13.8       150

    BRANDS               LIFESTAGE PREMIUM_CUSTOMER
0  natural   YOUNG SINGLES/COUPLES          Premium
1      ccs  MIDAGE SINGLES/COUPLES           Budget
2   smiths  MIDAGE SINGLES/COUPLES           Budget
3   smiths  MIDAGE SINGLES/COUPLES           Budget
4   kettle  MIDAGE SINGLES/COUPLES           Budget
```

19 Checking null

[40]:
```
data.isna().sum()
```

[40]:
```
DATE                0
STORE_NBR           0
LYLTY_CARD_NBR      0
TXN_ID              0
PROD_NBR            0
PROD_NAME           0
PROD_QTY            0
TOT_SALES           0
Packet_Wt           0
BRANDS              0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

[42]:
```
# Exporting the merged data
data.to_csv("QVI_data.csv", index=False)
```

[43]:
```
data["LIFESTAGE"].unique()
```

[43]:
```
array(['YOUNG SINGLES/COUPLES', 'MIDAGE SINGLES/COUPLES', 'NEW FAMILIES',
       'OLDER FAMILIES', 'OLDER SINGLES/COUPLES', 'RETIREES',
       'YOUNG FAMILIES'], dtype=object)
```

20 Interpreting No of sales by lifestage and premiumness of customer in the BAR

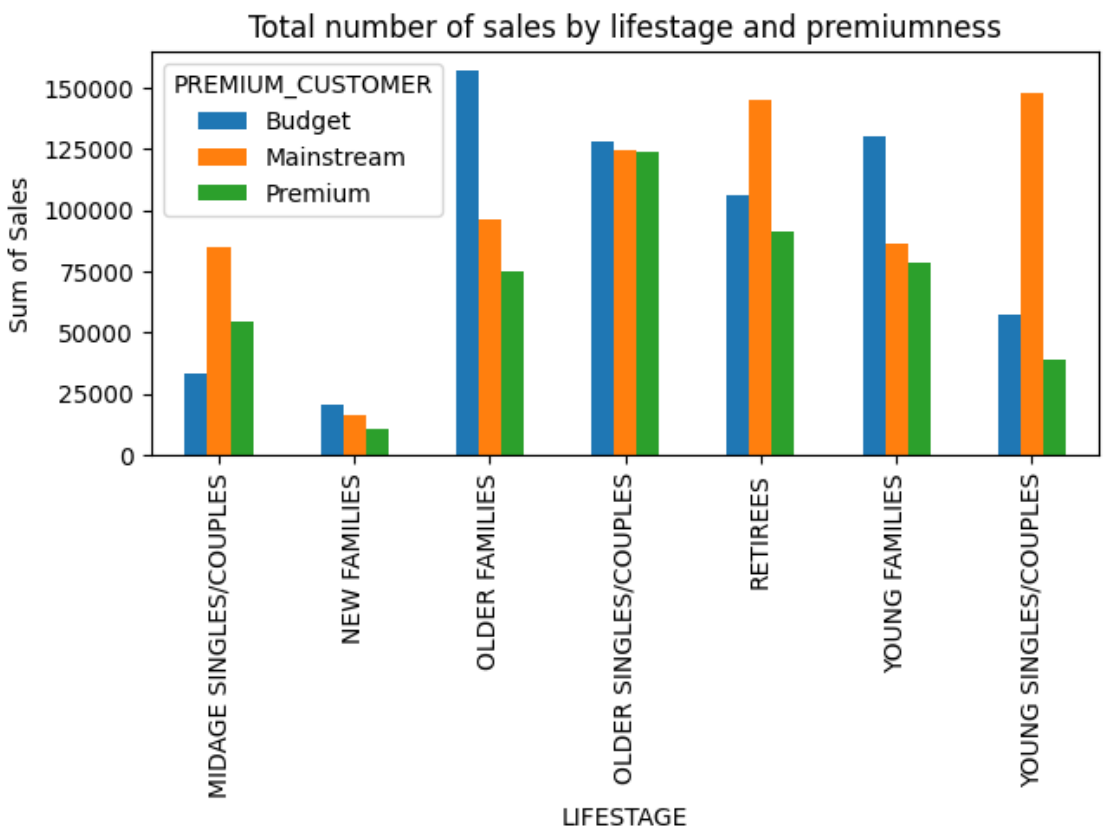
[58]:
```
sales_summary = data.groupby(["LIFESTAGE","PREMIUM_CUSTOMER"])["TOT_SALES"].
 ↪sum().unstack();
sales_summary
```

[58]:

PREMIUM_CUSTOMER	Budget	Mainstream	Premium
LIFESTAGE			
MIDAGE SINGLES/COUPLES	33345.70	84734.25	54443.85
NEW FAMILIES	20607.45	15979.70	10760.80
OLDER FAMILIES	156863.75	96413.55	75242.60
OLDER SINGLES/COUPLES	127833.60	124648.50	123537.55
RETIREES	105916.30	145168.95	91296.65
YOUNG FAMILIES	129717.95	86338.25	78571.70
YOUNG SINGLES/COUPLES	57122.10	147582.20	39052.30

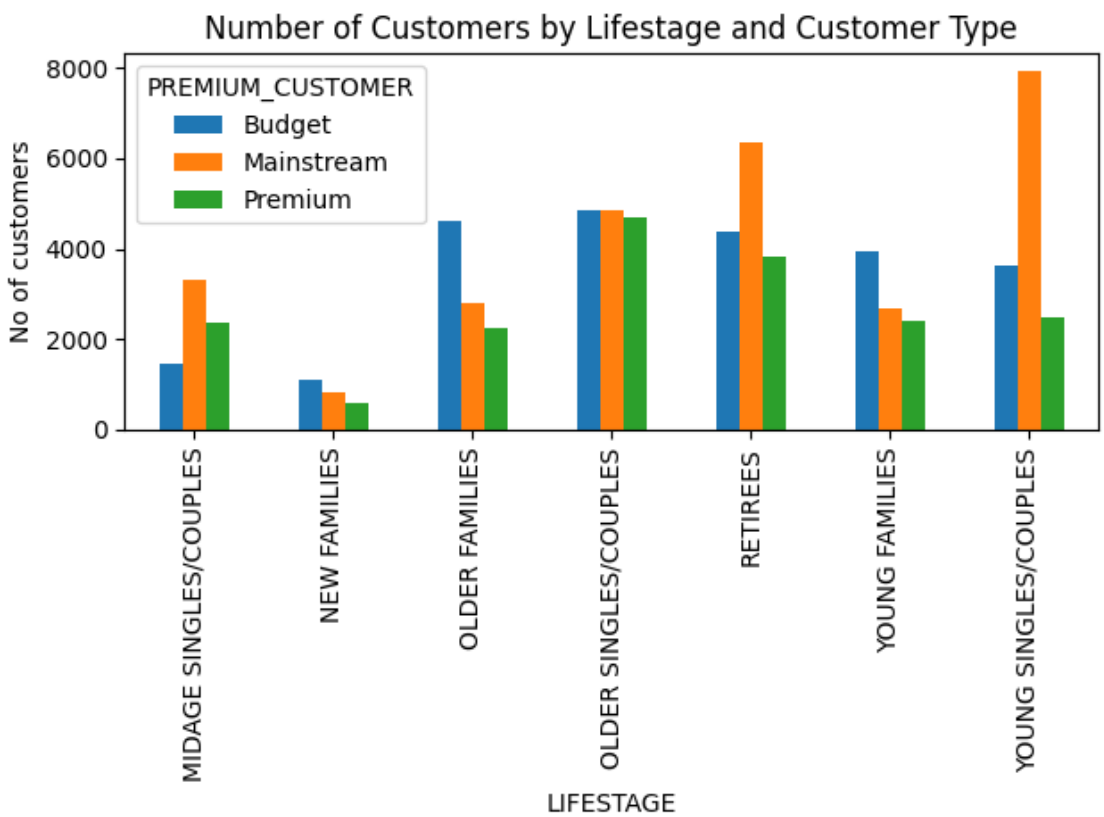
[69]:

```
#Interpreting on Bar
sales_summary.plot(kind="bar");
plt.tight_layout();
plt.title("Total number of sales by lifestage and premiumness")
plt.ylabel("Sum of Sales");
```



21 Interpreting no of customers on each categories of lifestage and premiumness on Bar

[84]:
```
unique_customers = data[["LYLTY_CARD_NBR" ,"LIFESTAGE" ,"PREMIUM_CUSTOMER"]].
↪drop_duplicates()
customer_count = unique_customers.groupby(["LIFESTAGE" ,␣
↪"PREMIUM_CUSTOMER"])["LYLTY_CARD_NBR"].count().unstack();
customer_count
customer_count.plot(kind="bar");
plt.ylabel("No of customers");
plt.title('Number of Customers by Lifestage and Customer Type');
plt.tight_layout();
plt.show()
```

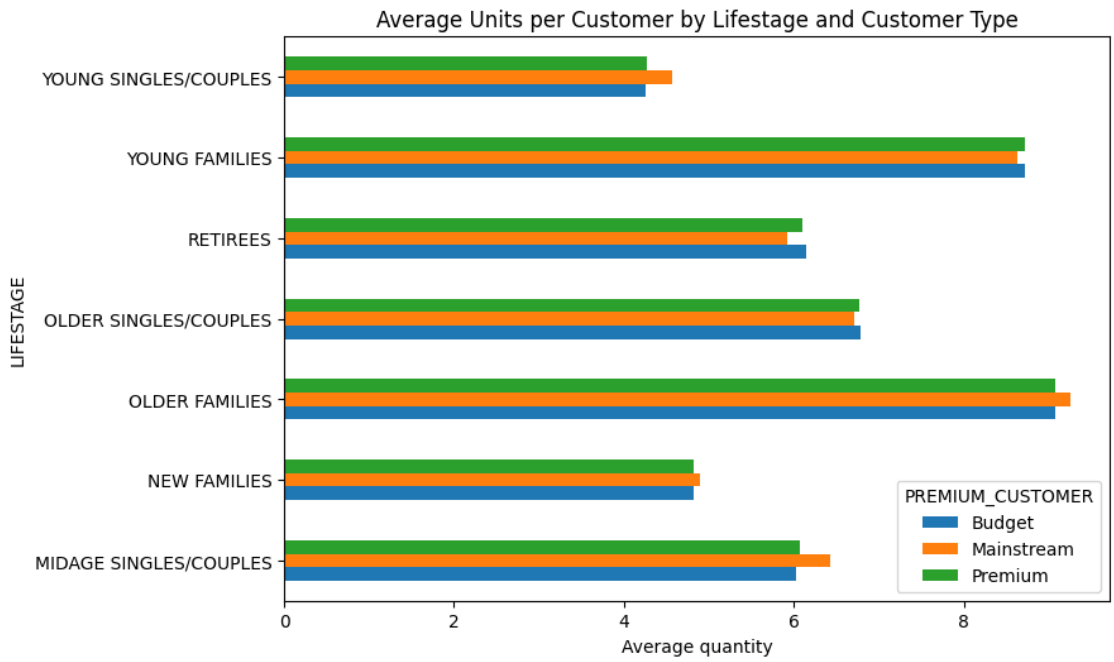


22 Average no of quantity buy by each life stage and premiumness.

[111]:
```
total_units_per_customer = data.groupby(['LYLTY_CARD_NBR', 'LIFESTAGE',␣
↪'PREMIUM_CUSTOMER'])['PROD_QTY'].sum().reset_index()
```

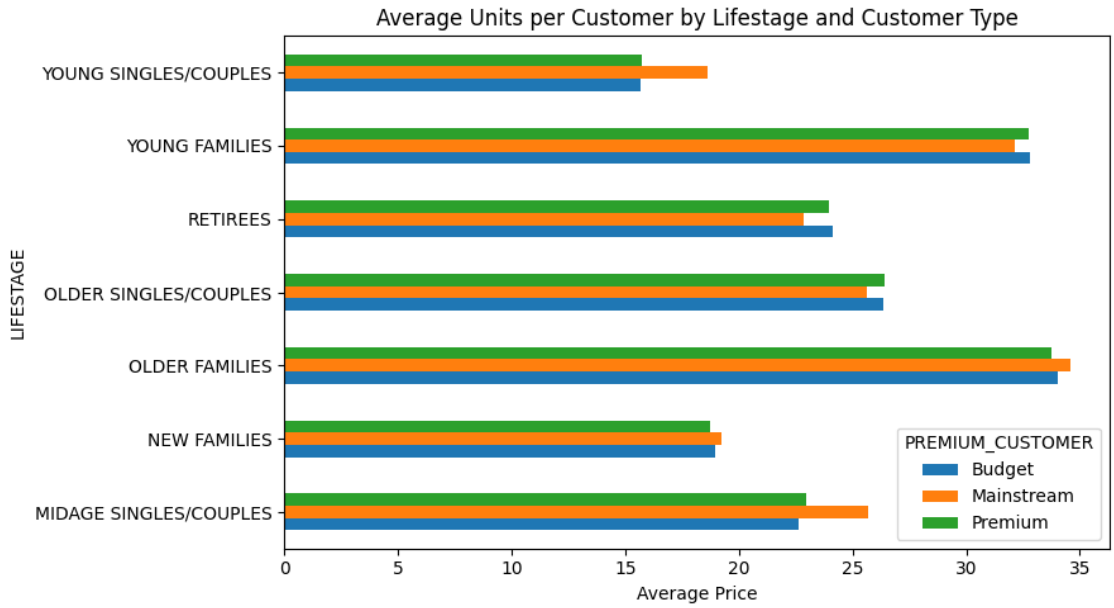
```
avg_units  = total_units_per_customer.groupby(['LIFESTAGE',␣
↪'PREMIUM_CUSTOMER'])['PROD_QTY'].mean().unstack()
avg_units.plot(kind="barh" , figsize = (9,5));
plt.tight_layout();
plt.title("Average Units per Customer by Lifestage and Customer Type");
plt.xlabel('Average quantity')
```

[111]: Text(0.5, 25.722222222222214, 'Average quantity')



23 Average price per customer lifestage and type.

[123]:
```
total_price_per_customer = data.groupby(["LYLTY_CARD_NBR" , "LIFESTAGE",␣
↪"PREMIUM_CUSTOMER"])["TOT_SALES"].sum().reset_index();
avg_price = total_price_per_customer.groupby(["LIFESTAGE" ,␣
↪"PREMIUM_CUSTOMER"])["TOT_SALES"].mean().unstack();
avg_price
avg_price.plot(kind = "barh", figsize=(9,5));
plt.title("Average Units per Customer by Lifestage and Customer Type");
plt.xlabel("Average Price");
plt.tight_layout();
```



24 T-Testing.

[125]:
```
data["UNIT_PRICE"] = data["TOT_SALES"] / data["PROD_QTY"]

unit_price_per_customer = data.groupby(
    ["LYLTY_CARD_NBR", "LIFESTAGE", "PREMIUM_CUSTOMER"]
)["UNIT_PRICE"].mean().reset_index()

mainstream = unit_price_per_customer[
    (unit_price_per_customer["PREMIUM_CUSTOMER"] == "Mainstream") &
    (unit_price_per_customer["LIFESTAGE"].isin([
        "YOUNG SINGLES/COUPLES",
        "MIDAGE SINGLES/COUPLES"
    ]))
]["UNIT_PRICE"]

budget_premium = unit_price_per_customer[
    (unit_price_per_customer["PREMIUM_CUSTOMER"].isin(["Budget", "Premium"])) &
    (unit_price_per_customer["LIFESTAGE"].isin([
        "YOUNG SINGLES/COUPLES",
        "MIDAGE SINGLES/COUPLES"
    ]))
]["UNIT_PRICE"]

from scipy.stats import ttest_ind
```

```
t_stat, p_value = ttest_ind(mainstream, budget_premium, equal_var=False)

print("T-statistic:", round(t_stat, 4))
print("P-value:", round(p_value, 6))
```

```
T-statistic: 37.7061
P-value: 0.0
```

25 BRAND prefrence comparison.

[130]:
```
target = data[
    (data["PREMIUM_CUSTOMER"] == "Mainstream") &
    (data["LIFESTAGE"] == "YOUNG SINGLES/COUPLES")
]

others = data[
    ~(
        (data["PREMIUM_CUSTOMER"] == "Mainstream") &
        (data["LIFESTAGE"] == "YOUNG SINGLES/COUPLES")
     )
]

target_brand = target["BRANDS"].value_counts(normalize=True) #Normalize is used␣
 ↪to convert into percentage.

others_brand = others["BRANDS"].value_counts(normalize=True)

brand_comparison = pd.DataFrame({
    "Target_Segment": target_brand,
    "Others": others_brand
}).fillna(0)

brand_comparison["Difference in %"] = (
    brand_comparison["Target_Segment"] -
    brand_comparison["Others"]
)

brand_comparison.sort_values("Difference in %", ascending=False)
```

[130]:

BRANDS	Target_Segment	Others	Difference in %
kettle	0.196684	0.164809	0.031875
doritos	0.106222	0.087876	0.018346
pringles	0.118451	0.100297	0.018154
twisties	0.046050	0.037650	0.008400
tostitos	0.045538	0.037769	0.007769
tyrrells	0.031672	0.025630	0.006042

```
cobs              0.044208  0.038861         0.005347
infuzions         0.049222  0.044433         0.004789
grain             0.029472  0.025071         0.004401
thins             0.059660  0.056819         0.002841
dorito            0.015503  0.012676         0.002827
infzns            0.014736  0.012571         0.002165
cheezels          0.017704  0.018737        -0.001033
french            0.003991  0.005898        -0.001907
ncc               0.003735  0.005924        -0.002189
grnwves           0.003582  0.006153        -0.002572
snbts             0.003633  0.006624        -0.002991
sunbites          0.002916  0.006052        -0.003136
burger            0.003172  0.006611        -0.003439
woolworths        0.002865  0.006426        -0.003561
cheetos           0.008494  0.012153        -0.003659
ccs               0.011359  0.019054        -0.007695
natural           0.016424  0.025216        -0.008792
ww                0.021643  0.043562        -0.021918
rrd               0.044771  0.067985        -0.023215
smiths            0.098291  0.125143        -0.026852
```

[ ]:

```
```